Assignment 1: Loan Items Tracker (5% of course total)

- Due date is **5 Jun, 11:59pm**. Refer to Canvas for any updates.
- Late penalty is 10% of max per calendar day (0 to 24 hour period past due), max. 2 days late.
- The assignment might be graded automatically. Make sure that your code compiles without warnings/errors and produces the required output. Also use the file names and structures indicated. Deviation from that might result in 0 mark.
- This assignment is to be done individually. Do not show another student your code, do not copy code from another person/online. Post all questions on Canvas (without your solution).
- You may use general ideas you find online and from others, but your solution must be your own.
- Your code MUST compile and run in VS Code with Oracle JDK 21, or marks will be deducted. Unless otherwise specified, your code MUST be created as a Java Project with "No build tools".

Description

In this assignment you are going to implement a text-based query system of loan items. The program will allow a user to maintain a list of loan items recording a small collection of their attributes.

Overall Requirements

You must have (at least) the following three classes (in separate .java files):

- A class for holding loan information: name (String), due date (LocalDate), publisher (String), and loaned-to (String). You can assume each string attribute will not be more than a String can hold.
    - Name/publisher/loaned-to are single-lines and may contain more than one word (e.g., "Intro To Java", "SFU", "John Smith"). **Only publisher can be empty**.
    - This class must correctly implement (override) toString(), such that when the object is passed to System.out.println() as a parameter its attributes will be displayed properly.
- A class for a text menu. This class handles all the Text UI features.
    - Have a field to store the menu's title (String). Come up with something fun!
    - Have a method to display (print) the menu to the screen.
        - Your application must automatically place a rectangle of #'s around the menu's title, sized to the length of the title. This must be computed, not hard-coded!
        - Follow the title banner with **the current system date**.
        - Automatically number the options starting at 1.
    - Have a method to read user's choice of option and carry out the corresponding action.
        - This method should in turn call other methods to carry out each action.
        - This method must be able to handle invalid inputs of the expected type. See the requirements below for details.
- A class for the main application. This class serves at the entry point of the program.
    - Contains the main() method which greets the user, then goes into the query loop.
    - Shows the user the available options, then asks the user for an input and handles it accordingly, repeat if needed.
    - Thanks and says goodbye to the user when they decide to quit the program.
    - Uses an ArrayList of loan items to maintain the list, saves it to a file upon quitting, and loads it from the file next time the program runs again.
    - You can make members in this class static so you just need to use them directly.

For this assignment, put all your classes in one package: **cmpt213.assignment.loanitemstracker**.

Your program must be able to save the list in JSON format (e.g., a JSON array object). When the system runs for the first time, there is no JSON file to load and this list is empty (because nothing has been added yet). When the user exits the system, your system automatically generates a JSON file (list.json) and saves it to your project folder (use a hard-coded relative path starting with **./**). When the user runs the system again (and subsequently), this JSON file will be loaded and used to populate the list. When the user exits again, this file will be overwritten with the new information. Both the loading and saving are done automatically by the application – the user doesn't need to do anything to trigger that.

Text Interface Requirements

- When the system prompts the user to choose a menu option, if the user enters an invalid number, it must re-ask the user to enter a valid value.
    - You may assume the user always enters correct type of data: when asked for an int, it is OK if the application crashes when the user enters a non-int such as 'A'.
- **Main Menu Option 1: List all items**
    - For each loan item, first show the item number, then show its name and its publisher on the next line, to whom it is loaned in another line, and its due date info on another line.
        - The item number is an automatically generated counter (starting with 1).
        - The list is ordered by dates (oldest first).
    - If the list is empty, display "No items to show." instead.
- **Main Menu Option 2: Add an item**
    - Ask the user for name, due date, publisher, and loaned-to. Handle cases when an invalid value is inputted (e.g., negatives, non-existing date/time).
        - Date is specified by year, month, and day, use 1900 as the minimal year.
        - To check if a date is valid, look up the "LocalDate.of" static method.
        - You don't have to check for duplicates.
    - When done, display an acknowledgement message. For example, if "Intro to Java" is added, display "Intro to Java has been added to the list.".
- **Main Menu Option 3: Remove an item**
    - List all loan items currently in the system along with their automatically generated item number. Ordered by dates (oldest first).
        - If the list is empty, display "No items to show." instead.
    - Otherwise, allow the user to select a loan item (by number), or 0 to cancel.
    - Entering an invalid number (like -3, or a value more than the number of items) needs to be handled by the application (tell the user that it is invalid and ask for a number again). Entering invalid data type (e.g., "hello") does not need to be handled.
    - When done, display an acknowledgement message. For example, if "OOD Principles" is removed, display "OOD Principles has been removed from the list.".
- **Main Menu Option 4: List overdue items** (up to and including yesterday, exclude the rest)
    - Show a list of all items due before today, in the same format as Option 1.
    - If the list is empty, display "No items to show." instead.
- **Main Menu Option 5: List upcoming items** (including today and after, exclude the rest)
    - Show a list of all items due today and after, in the same format as Option 1.
    - If the list is empty, display "No items to show." instead.

- **Main Menu Option 6: Exit**
    - Display the message "Saving the list to ./list.json", followed by "Thank you for using Loan Items Tracker!" or something fun, then exit the application.
    - It will also automatically save the list in JSON format to the file.

Your text UI need not match the sample exactly, but it should be of equal quality and include all the required information (e.g., today's date in the menu, number of overdue/due-in days).

Coding Requirements

- As an exercise, **Options 4&5 must be done with Stream API**. No for-/while-loops are allowed.
- Your code must conform to the programming style guide for this course. See course website.
- All classes must have both class- and method-level JavaDoc comments describing the purpose of the class and methods. Field-level comments are optional. **Generate them in a folder called docs**.

Suggestions

- Think about the design before you start coding.
    - List the classes you expect to create.
    - For each class, decide what its responsibilities will be.
    - Think through some of the required features. How will each of your classes work to implement this feature? Can you think of design alternatives?
- Write your code in progressing level of details.
    - Your code does not need to be fully functional at the firs time. Start with just printing a message in a method to have the logic correct.
    - Use refractoring to improve your code in later stages.
- You can reuse some of the methods to save time. For example, think about how to reuse the "List all expenses" option in the "Remove an expense" option.

Submission Instructions

- Submit a ZIP file of your project to the corresponding folder on Canvas. Name it like this: **John_Smith_012345678_Assignment1.zip** and replace the name with yours. See Canvas for how to on create and test your ZIP file for submission. **Deviation might result in mark deductions**. If you have any difficulties in submitting your file on Canvas, email it to the instructor.
- Email a copy to yourself at your SFU account before the deadline for save keeping.
- Any libraries you use must be included in the project as well (under the lib folder).
- All submissions might automatically be compared for unexplainable similarities.

Useful Resources

- You MUST use this GSON library (v2.13.2) for handling JSON: https://github.com/google/gson
    - Read in particular the toJson and fromJson methods
    - Download the .jar file via the "Gson jar downloads" link and put this file in the lib folder of your Java project – this is how you add a library in VS Code without any build tools.
    - Optional: take a look at methods in GsonBuilder to make your json file look nicer
- A video tutorial for File I/O with GSON by Dr. Brian Fraser: https://youtu.be/HSuVtkdej8Q
- Java API of the LocalDate class: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/LocalDate.html

Notes

The GSON library does not natively support reading/writing LocalDate objects. To make it work you will have to define how the operations are done when creating the Gson object for reading/writing. Here is the code snippet before calling the toJson and fromJson methods from a Gson object.

```
Gson myGson = new GsonBuilder().registerTypeAdapter(LocalDate.class,
                new TypeAdapter<LocalDate>() {
                  @Override
                  public void write(JsonWriter jsonWriter,
                    LocalDate localDate) throws IOException {
                      jsonWriter.value(localDate.toString());
                  }

                  @Override
                  public LocalDate read(JsonReader jsonReader) throws IOException {
                    return LocalDate.parse(jsonReader.nextString());
                  }
                }).create();
```

This code constructs a custom Gson object that allows additional configurations not available by default. The same manner can be used to make your json file look nicer (optional). For more details, refer to: https://www.javadoc.io/doc/com.google.code.gson/gson/latest/com.google.gson/com/google/gson/GsonBuilder.html

Sample Input/Output

(first time running the program, system date is May 23, user input is highlighted for clarity)*

```
######################
# Loan Items Tracker #
######################
Today is: 2026 May 23
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 7
Invalid selection. Enter a number between 1 and 6
Choose an option by entering 1-6: 12
Invalid selection. Enter a number between 1 and 6
Choose an option by entering 1-6: 1
No items to show.

######################
# Loan Items Tracker #
######################
Today is: 2026 May 23
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
```



```
6: Exit
Choose an option by entering 1-6: 2
Enter the name of the loan item: Intro to Java
Enter the year of the due date (e.g., 2026): 2026
Enter the month of the due date (1-12): 5
Enter the day of the due date (1-28/29/30/31): 22
Enter the publisher of the loan item: SFU
Enter the name to which the item is loaned: John Smith
Intro to Java has been added to the list.

######################
# Loan Items Tracker #
######################
Today is: 2026 May 23
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 1

#1
Intro to Java
- published by SFU
- loaned to John Smith
- due on 2026-05-22 (overdue by 1 day(s))

######################
# Loan Items Tracker #
######################
Today is: 2026 May 23
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 6
Saving the list to ./list.json...

Thanks for using Loan Items Tracker!
```

(second time running the program, system date is May 24, user input is highlighted for clarity)*

```
######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
```



```
Choose an option by entering 1-6: 1

#1
Intro to Java
- published by SFU
- loaned to John Smith
- due on 2026-05-22 (overdue by 2 day(s))

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 2
Enter the name of the loan item: Design Patterns in C++
Enter the year of the due date (e.g., 2026): 1881
Error: year must be at least 1900.
Enter the year of the due date (e.g., 2026): 2026
Enter the month of the due date (1-12): 4
Enter the day of the due date (1-28/29/30/31): 30
Enter the publisher of the loan item:
Enter the name to which the item is loaned: Bruce Wayne
Design Patterns in C++ has been added to the list.

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 2
Enter the name of the loan item: Algorithms to Live By
Enter the year of the due date (e.g., 2026): 2026
Enter the month of the due date (1-12): 6
Enter the day of the due date (1-28/29/30/31): 31
Error: this date does not exist.
Enter the day of the due date (1-28/29/30/31): 30
Enter the publisher of the loan item: Penguin Canada
Enter the name to which the item is loaned: Clark Kent
Algorithms to Live By has been added to the list.

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
```



```
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 2
Enter the name of the loan item: The Design of Everyday Things
Enter the year of the due date (e.g., 2026): 2026
Enter the month of the due date (1-12): 5
Enter the day of the due date (1-28/29/30/31): 24
Enter the publisher of the loan item: Basic Books
Enter the name to which the item is loaned: Barry Allen
The Design of Everyday Things has been added to the list.

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 1

#1
Design Patterns in C++
- published by
- loaned to Bruce Wayne
- due on 2026-04-30 (overdue by 24 day(s))

#2
Intro to Java
- published by SFU
- loaned to John Smith
- due on 2026-05-22 (overdue by 2 day(s))

#3
The Design of Everyday Things
- published by Basic Books
- loaned to Barry Allen
- due on 2026-05-24 (due today)

#4
Algorithms to Live By
- published by Penguin Canada
- loaned to Clark Kent
- due on 2026-06-30 (due in -37 day(s))

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
```



```
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 4
#1
Design Patterns in C++
- published by
- loaned to Bruce Wayne
- due on 2026-04-30 (overdue by 24 day(s))
#2
Intro to Java
- published by SFU
- loaned to John Smith
- due on 2026-05-22 (overdue by 2 day(s))

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 5
#1
The Design of Everyday Things
- published by Basic Books
- loaned to Barry Allen
- due on 2026-05-24 (due today)
#2
Algorithms to Live By
- published by Penguin Canada
- loaned to Clark Kent
- due on 2026-06-30 (due in 37 day(s))

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 3

#1
Design Patterns in C++
- published by
- loaned to Bruce Wayne
```



```
- due on 2026-04-30 (overdue by 24 day(s))

#2
Intro to Java
- published by SFU
- loaned to John Smith
- due on 2026-05-22 (overdue by 2 day(s))

#3
The Design of Everyday Things
- published by Basic Books
- loaned to Barry Allen
- due on 2026-05-24 (due today)

#4
Algorithms to Live By
- published by Penguin Canada
- loaned to Clark Kent
- due on 2026-06-30 (due in 37 day(s))

Enter the item number you want to remove (0 to cancel): 5
Invalid selection. Enter a number between: 0 and 4
Enter the item number you want to remove (0 to cancel): 3
The Design of Everyday Things has been removed from the list.

######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 1

#1
Design Patterns in C++
- published by
- loaned to Bruce Wayne
- due on 2026-04-30 (overdue by 24 day(s))

#2
Intro to Java
- published by SFU
- loaned to John Smith
- due on 2026-05-22 (overdue by 2 day(s))

#3
Algorithms to Live By
- published by Penguin Canada
- loaned to Clark Kent
- due on 2026-06-30 (due in 37 day(s))
```



```
######################
# Loan Items Tracker #
######################
Today is: 2026 May 24
1: List all items
2: Add an item
3: Remove an item
4: List overdue items
5: List upcoming items
6: Exit
Choose an option by entering 1-6: 6
Saving the list to ./list.json...

Thanks for using Loan Items Tracker!
```

*The sample input/output is for demo purposes only. We'll use different test cases when grading.

Academic Honesty

It is expected that within this course, the highest standards of academic integrity will be maintained, in keeping with SFU's Policy S10.01, "Code of Academic Integrity and Good Conduct." In this class, collaboration is encouraged for in-class exercises and the team components of the assignments, as well as task preparation for group discussions. However, individual work should be completed by the person who submits it. Any work that is independent work of the submitter should be clearly cited to make its source clear. All referenced work in reports and presentations must be appropriately cited, to include websites, as well as figures and graphs in presentations. If there are any questions whatsoever, feel free to contact the course instructor about any possible grey areas.

Some examples of unacceptable behavior:

- Handing in assignments/exercises that are not 100% your own work (in design, implementation, wording, etc.), without a clear/visible citation of the source.
- Using another student's work as a template or reference for completing your own work.
- Sharing your work with or making your work available on any platform for anyone who would benefit from it (e.g., other students in the course).
- Using any unpermitted resources during an exam.
- Looking at, or attempting to look at, another student's answer during an exam.
- Submitting work that has been submitted before, for any course at any institution.

All instances of academic dishonesty will be dealt with severely and according to SFU policy. This means that Student Services will be notified, and they will record the dishonesty in the student's file. Students are strongly encouraged to review SFU's Code of Academic Integrity and Good Conduct (S10.01) available online at: http://www.sfu.ca/policies/gazette/student/s10-01.html.

Use of ChatGPT or Other AI Tools

As mentioned in the class, we are aware of them. I see them as helpers/tutors from which you can look for inspiration. However, these tools are not reliable and they tend to be overly confident about their answers, sometimes even incorrect. It is also very easy to grow a reliance to them and put yourself at risk of not actually learning anything and even committing academic dishonesty. Other issues include:

- When it comes to uncertainty, you won't know how to determine what is correct.
- If you need to modify or fine tune your answer, you won't know what to do.
- You will not be able to learn the materials and thus will not ne able to apply what you learn in situations where no external help is available, for example, during exams and interviews.

For introductory level courses and less sophisticated questions, it is likely that you'll get an almost perfect answer from these tools. But keep in mind if you can get the answer, everyone can also get the answer, and using the answer given by these tools as yours is considered academic dishonesty as you are claiming work that is not done by you as yours. You can, however, ask general questions like "how do I write a for-loop?", "explain mergesort to me" and use the answers to help you come up with your own answers – make sure you have cited it at the top of your code/essay as comments by stating what tool you used what questions you asked, and how you used the answers. **In particular, do not copy & paste the questions, in part or in whole, and have these tools give you the answers; or copy & paste your solutions, in part or in whole, and have these tools to "fix/optimize" for you.**

Note that different instructors might have different policies regarding the use of these tools. Check with them before you proceed with assignments from other courses.

